

Introduction to Object Oriented Programming

Created by <Academia de Código_>

Object Oriented Programming

Why?

An orange square icon with rounded corners containing the white text "{oop}" in a monospaced font.

- Handle size and complexity of software
- Produce reusable and maintainable software

Case Study



In the early 1970s, the US DoD commissioned a task force

Their mission was to investigate why the IT budget always went out of control without much to show for...

- 80% of the budget went to the software, while the remaining 20% was spent on hardware
- More than 80% of the software budget went to maintenance, only the remaining 20% was used on new software development
- Hardware components could be shared and reused on other products
- Software was often non-sharable and non reusable
- Most often, the integrity of hardware did not affect other products, faults were isolated
- Software faults could affect other running programs

The task force proposed to make software behave like hardware

Object Oriented Programming

How?

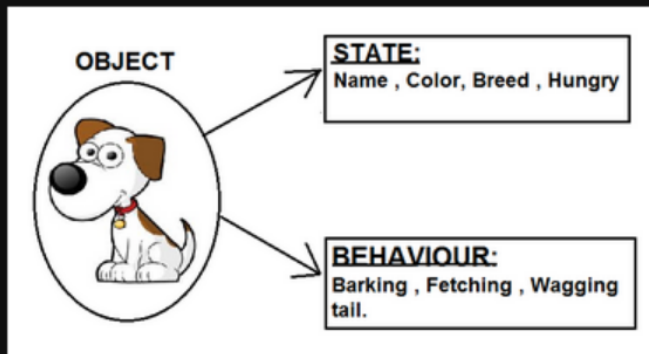


DoD replaced over 450 computer languages which were used to build DoD systems with an object-oriented language called Ada

Objects

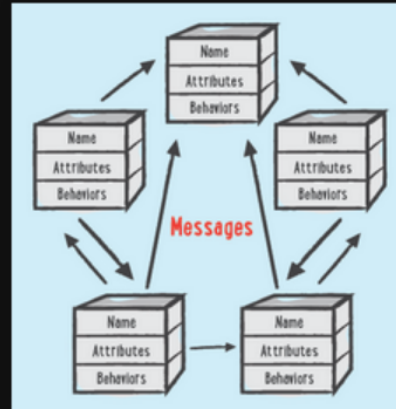
Software entities that encapsulate state and behaviour

- State is what the object knows about itself (attributes)
- Behaviour is what the object can do (methods)



OOP Program

An object oriented program consists of many objects interacting with each other by sending messages



In Java, invoking a method on another object is equivalent to passing it a message

A calculator object



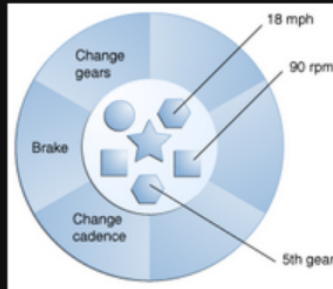
What is a calculator?

How we define an Object

- What it looks like (e.g. shape, size, color, elements, etc.);
- What it does (e.g. adds, subtracts, multiplies, etc.).

An Object in Java

An object in Java is a container of properties and related behaviours that operate on those properties.



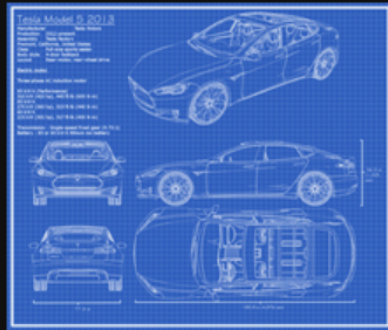
How do we create objects?



We need a set of specifications!

Classes and Instances

In Java, that set of specifications is called a Class and each object is an Instance of that Class.



Class



Instance



Instance



Instance



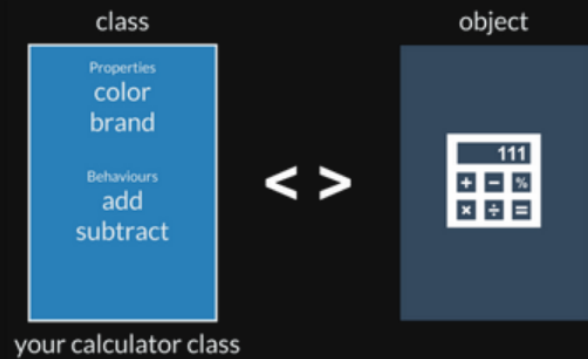
Instance

each class is a new data type in the program

A calculator Class and Instances



Calculator Class



{ Hands On }

Calculator Class

```
/**
 * Calculator.java
 */

// Create a new class
public class Calculator {

    // properties or instance variables
    public String brand;
    public String color;

}
```

{ Hands On }

Create Calculator Instances

```
/**
 * Main.java
 */

public static void main(String[] args) {

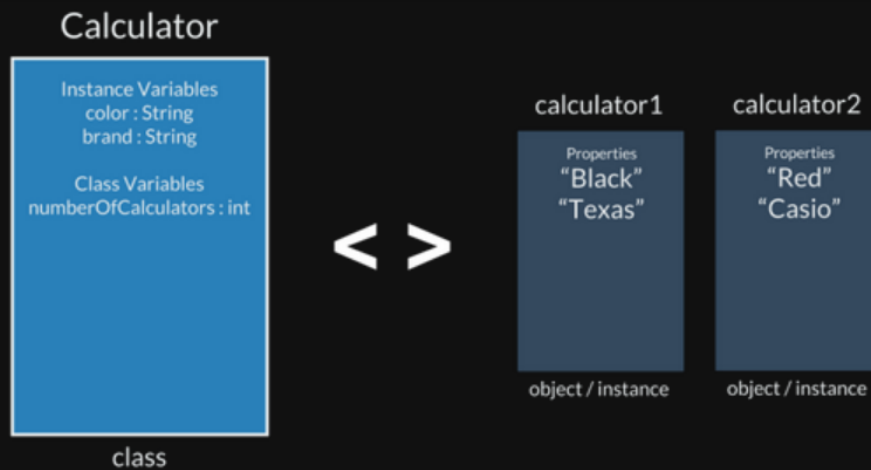
    // create a Calculator instance
    Calculator calculator = new Calculator();

    // assign values to the instance variables
    calculator.brand = "Casio";
    calculator.color = "Grey";

    // a new Calculator instance with different values assigned
    Calculator calculator1 = new Calculator();
    calculator1.brand = "Texas Instruments";
    calculator1.color = "Black";

    // output the result
    System.out.println("Calculator is a " + calculator.brand + " of color " + calculator.color);
}
```

Classes and Objects



Methods

A method is a function defined in a class. It consists of a group of statements that perform a specific task

In OOP, methods are the way to provide functionality to our objects

{ Hands On }

Declaring a method

```
/**
 * Calculator.java
 */

public class Calculator {

    public String brand;
    public String color;

    // declare the method add
    public void add() {

        int number1 = 1;
        int number2 = 2;

        // print the result
        System.out.println(number1 + number2);
    }
}
```

{ Hands On }

Calling a method

```
/**
 * Main.java
 */

public static void main(String[] args) {

    // create a Calculator
    Calculator calculator = new Calculator();

    // assign values to the properties
    calculator.brand = "Casio";
    calculator.color = "Grey";

    // call the method
    calculator.add();

    // prints ALWAYS 3
}
```

Why should I want a method that only adds 1 and 2?

{ Hands On }

Adding parameters to a method

```
/**
 * Calculator.java
 */
public class Calculator {

    public String brand;
    public String color;

    // declare the method add with two parameters
    // method signature: name + parameters
    public void add(int number1, int number2) {

        // this makes no sense
        // int number1 = 1;
        // int number2 = 2;

        // print the result
        System.out.println(number1 + number2);
    }
}
```

We must always conform to the method signature!

{ Hands On }

Calling a method with parameters

```
/**
 * Main.java
 */

public static void main(String[] args) {

    // create a Calculator
    Calculator calculator = new Calculator();

    // assign values to the properties
    calculator.brand = "Casio";
    calculator.color = "Grey";

    // call the method add with parameters
    calculator.add(3, 4);

    // prints the result according to the parameters, in this case 7
}
```

What about the void?

```
/**
 * Calculator.java
 */
public class Calculator {
    public String brand;
    public String color;

    public void add(int number1, int number2) {
        System.out.println(number1 + number2);
    }
}
```

The keyword void tells the compiler the method will not return anything to its caller.

Return type

```
/**
 * Calculator.java
 */

public class Calculator {

    public String brand;
    public String color;

    // this method returns an int
    public int add(int number1, int number2) {

        return number1 + number2;
    }
}
```

The return type defines the data type the method returns when called.

Using the return value

```
/**
 * Main.java
 */

public static void main(String[] args) {

    Calculator calculator = new Calculator();

    calculator.brand = "Casio";
    calculator.color = "Grey";

    // get the return value
    int result = calculator.add(3, 4);

    result = calculator.add(result, 5);

    System.out.println(result);
}
```

Method Overloading

Feature that allows a class to have two or more methods with the same name if their argument lists are different

```
public class Calculator {  
  
    // Method for performing the sum operation with 2 integers  
    public int add(int number1, int number2) {  
        // ...  
    }  
  
    // Method for performing the sum operation with 2 floats  
    public float add(float number1, float number2) {  
        // ...  
    }  
  
}
```

The method name, number of arguments and their types constitute the method signature.

Constructors

A constructor is a method called whenever we create a new object

It is responsible for initializing the object

Default Constructor

```
/**
 * Main.java
 */
public static void main(String[] args) {

    Calculator calculator = new Calculator(); // using the default constructor

    calculator.brand = "Casio";
    calculator.color = "Grey";

    int result = calculator.add(3, 4);

    System.out.println(result);
}
```

Declaring a new Constructor

```
/**
 * Calculator.java
 */

public class Calculator {

    public String brand;
    public String color;

    // declaring a new constructor
    public Calculator(String brand, String color) {
        this.brand = brand;
        this.color = color;
    }

    // this method returns an int
    public int add(int number1, int number2) {
        return number1 + number2;
    }
}
```

Using the new Constructor

```
/**
 * Main.java
 */

public static void main(String[] args) {

    // using the new constructor
    Calculator calculator = new Calculator("Casio", "Grey");

    // calculator.brand = "Casio";
    // calculator.color = "Grey";

    int result = calculator.add(3, 4);

    System.out.println(result);
}
```

If a constructor is explicitly defined, the no-argument default constructor is not generated anymore.

Encapsulation

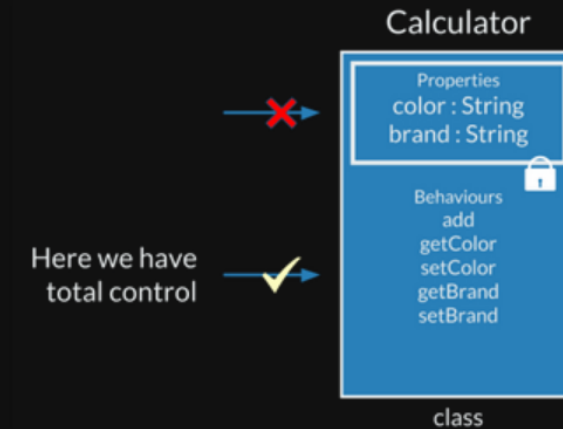
Encapsulation refers to hiding data and implementation from the outside and providing methods to its access and use.

Why Encapsulation?



- Provides an interface for the object behaviour;
- Improves maintainability, flexibility and reusability;
- Allows to implement read-only/write-only access.

Getters and Setters



- Getter: returns the instance variable;
- Setter: sets the new value to the instance variable.

Hiding the data

```
/**
 * Calculator.java
 */

public class Calculator {

    // hiding the data from the "outside"
    private String brand;
    private String color;

    public Calculator(String brand, String color) {
        this.brand = brand;
        this.color = color;
    }

    // GETTER
    public String getBrand() {
        return this.brand;
    }

    // SETTER
    public void setBrand(String brand) {
        this.brand = brand;
    }
}
```

Accessing the data

```
/**
 * Main.java
 */

public static void main(String[] args) {

    // using the new constructor
    Calculator calculator = new Calculator("Casio", "Grey");

    // using the SETTER
    calculator.setBrand("Texas Instruments");

    // using the GETTER
    String brand = calculator.getBrand();

    int result = calculator.add(3, 4);

    System.out.println(result);
}
```

Static Properties

We can create variables that are common to all instances of a class. They are associated with the class, rather than with a specific object.

We call them static or class variables and changing their value in one object will change its value for all objects

Instance vs Class variables

```
/**
 * Calculator.java
 */

// Create a new class
public class Calculator {

    // class variables: common to all instances
    public static int calculatorsManufactured;

    // fields or instance variables: unique to each single instance
    private String brand; // Casio, Texas Instruments
    private String color; // Grey, Black
}
```

A public class variable is the equivalent of a Global variable

Global variables break the encapsulation principle and should be used carefully!

Accessing Static Variables

```
/**
 * Main.java
 */

public static void main(String[] args) {

    // create a Calculator instance
    Calculator calculator = new Calculator();

    // ...

    // class variables are referenced by the class name itself
    System.out.println(Calculator.calculatorsManufactured);
}
```

Static Methods

```
/**
 * Calculator.java
 */
public class Calculator {

    // class variables: common to all instances
    private static int calculatorsManufactured;

    // increment the number of calculators manufactured in the constructor
    public Calculator(String brand, String color) {
        this.brand = brand;
        this.color = color;
        calculatorsManufactured++;
    }

    // class method: obtain the total number of calculators manufactured
    public static int getCalculatorsManufactured() {
        return calculatorsManufactured;
    }
}
```

Can NOT access instance variables!

Using Static Methods

```
/**
 * Main.java
 */

public static void main(String[] args) {

    // class methods are invoked with the class name
    System.out.println(Calculator.getCalculatorsManufactured());

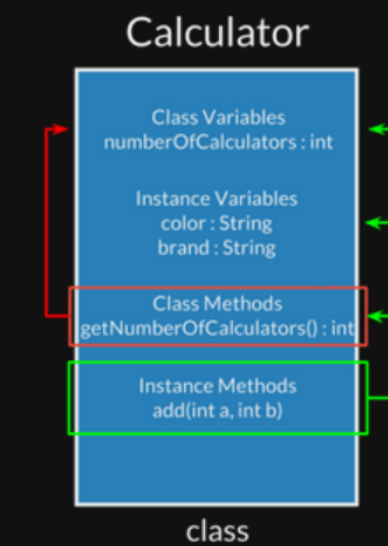
}
```

Static methods are globally available

They do not require passing object references around, but...

They break the solid principles of OOP design and should be used carefully!

Instance and Static Methods



{ Exercise }

Guess a Number



- System chooses a number
- Each player will in turn try to guess the number
- Game ends when one of the players makes the correct guess
- Start simple, no input, all guesses are random between 0 and a max value

Enumerations

Simply a set of possible values

No Enumerations

```
int currentDrink = 0;
switch(currentDrink) {
    case 0: // Beer
        System.out.println("Drinking beer.");
        break;
    case 1: // Vodka
        System.out.println("Drinking vodka.");
        break;
    case 2: // Water
        System.out.println("Drinking water.");
        break;
}
```

Confusing... And what if we set currentDrink = 10?

There must be a better and safer way...

Constants?

```
public class Drink {  
    public final static int BEER = 0;  
    public final static int VODKA = 1;  
    public final static int WATER = 2;  
}  
  
int currentDrink = Drink.BEER;  
  
switch (currentDrink) {  
    // ...  
}
```

Looks nicer, but still not safe:

```
// Does this make any sense?  
int currentDrink = Drink.BEER * Drink.VODKA;  
  
switch (currentDrink) {  
    // ...  
}
```

Java Enumerations

An enum is a class that has a predefined set of instances

```
/**
 * DrinkType.java
 */

public enum DrinkType {
    BEER,
    VODKA,
    WATER
}

// DrinkType is a java enumerated type
DrinkType currentDrink = DrinkType.BEER;

switch(currentDrink) {

    case BEER:
        System.out.println("Drinking beer.");
        break;

    case VODKA:
        System.out.println("Drinking vodka.");
        break;

}
```

Why Enumerations?

- Type safety (enforced by the compiler)
- Behaviour (just like ordinary classes)
- Readability (DrinkType.BEER instead of 0)
- Correctness (Input limited to predefined set of values)

When to use them?

Should always be used when a variable (especially a method parameter) can only take one out of a small set of possible values

Enum creation

```
/**
 * Operation.java
 */

public enum Operation {
    ADD,
    SUBTRACT,
    MULTIPLY,
    DIVIDE
}
```

ADD, SUBTRACT, MULTIPLY and DIVIDE are actually constant references to 4 Operation type instances

Using the Enum

The Enum is a new data type

```
/**
 * Calculator.java
 */

public class Calculator {

    private String brand;
    private String color;

    // private int operation;
    private Operation operation;

}
```

```
/**
 * Main.java
 */

public static void main(String[] args) {

    calculator.setOperation(Operation.ADD); // instead of calculator.setOperation(0);

}
```

Enum Properties, Constructors and Instance Methods

```
public enum Operation {  
    ADD("+"), // Each enum value is actually a constructor call  
    SUBTRACT("-"),  
    MULTIPLY("x"),  
    DIVIDE("/"); //end with ; when declaring class members afterwards  
  
    private String symbol;  
  
    Operation(String symbol) {  
        this.symbol = symbol;  
    }  
  
    public String getSymbol() {  
        return this.symbol;  
    }  
}
```

```
public static void main(String[] args) {  
  
    Operation add = Operation.ADD;  
    System.out.println(add.getSymbol());  
}
```

Enum Class Methods

```
public enum Operation {  
  
    public static String showOperations() {  
        String result = "Available Operations: ";  
  
        // Operation.values() returns an array with all the enum values  
        for(Operation operation : Operation.values()) {  
            result += operation.symbol + " ";  
        }  
  
        return result;  
    }  
}  
  
/**  
 * Main.java  
 */  
  
public static void main(String[] args) {  
  
    System.out.println(Operation.showOperations()); // Available Operations: + - x /  
}
```

{ Exercise }

Rock-Paper-Scissors

- Player with more wins after a set of rounds wins
- Each player will in turn select a hand randomly
- No input is required

